

大营销平台（抽奖）业务篇

业务全流程梳理

- 用户端UI图
- 运营端界面
- 策略领域
 - 装配策略（子域）
 - 策略抽奖（子域）
 - 奖品库存扣减方案设计
- 抽奖活动领域
 - 用户参与抽奖活动（参与抽奖子域）
 - 抽奖活动充值（额度子域）
 - 活动装配子域（预热活动）
 - 活动领域下各子域分析
- 发奖领域
- 跨领域核心逻辑
 - 抽奖
- 返利领域
- 积分领域

本文是大营销平台学习后画图梳理以及面向面试思考技术方案和难点的文档，面试前查阅即可。

业务全流程梳理

用户端UI图

大营销平台 - 抽奖展示

营销会员卡

id: xiaofuge

我的积分: 130.91 ¥

已签

2024年06月22日

抽奖次数: 1188

刷新

装配抽奖「测试前点击预热数据」

150次抽奖

200 ¥

兑换

50次抽奖

100 ¥

兑换

5次抽奖

20 ¥

兑换

1次抽奖

5 ¥

兑换



抽奖阶梯1

27/70

抽奖阶梯2

10/10

抽奖阶梯3

27/1000

必中奖品范围

1. 轻霸办公椅
2. 小霸王游戏机

必中奖品范围

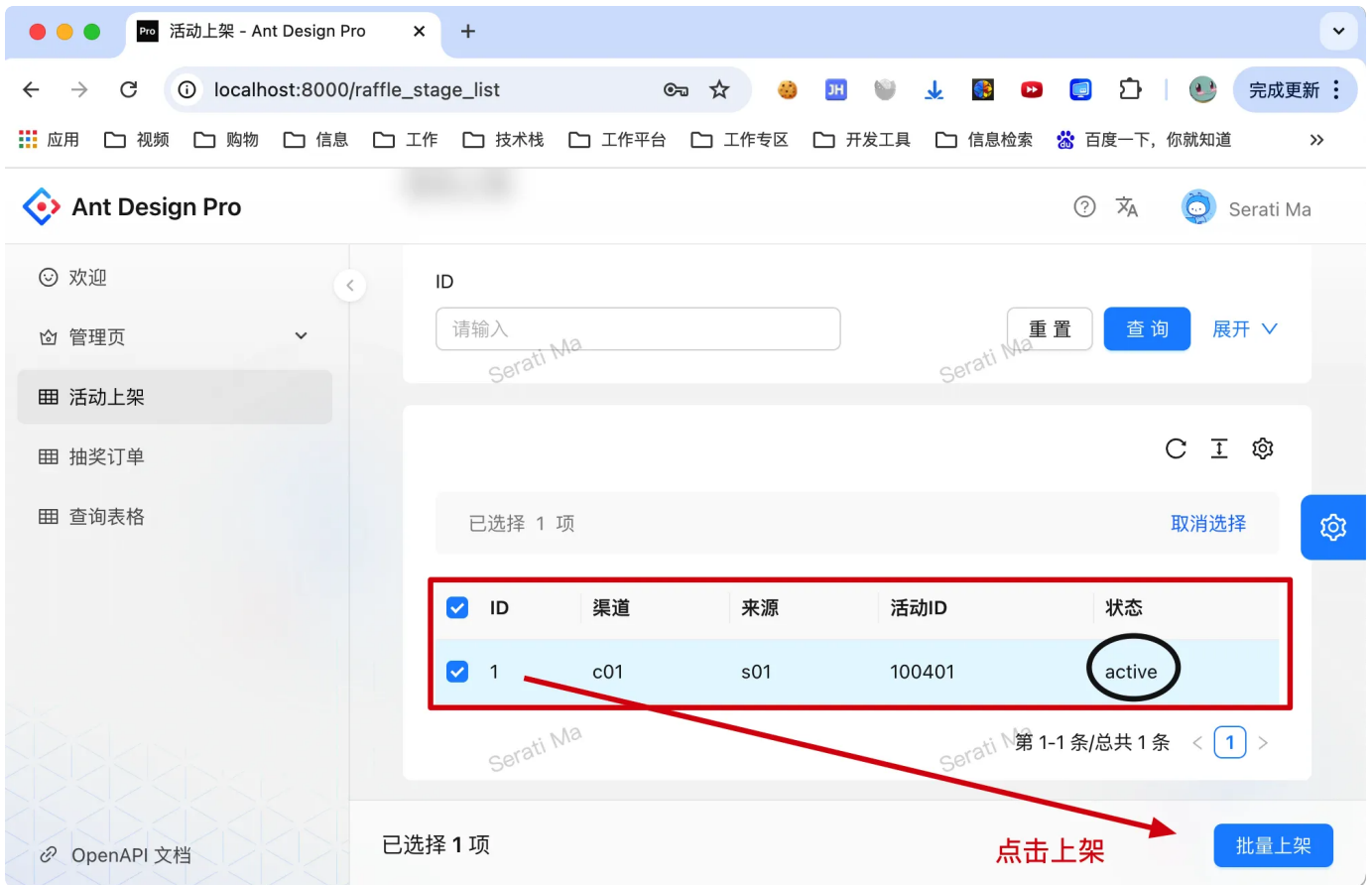
1. OpenAI会员卡
2. 支付优惠券

必中奖品范围

1. 小米台灯
2. 小米su7周体验

本项目为 星球「码农会锁」第8个实战项目 <https://gaga.plus> @小傅哥

运营端界面



策略领域

装配策略（子域）

装配策略主要做两件事：

1. 装配策略下奖品的库存（Redis）
2. 装配概率值和奖品的映射关系（Redis） 默认是 $O(1)$ 时间复杂度

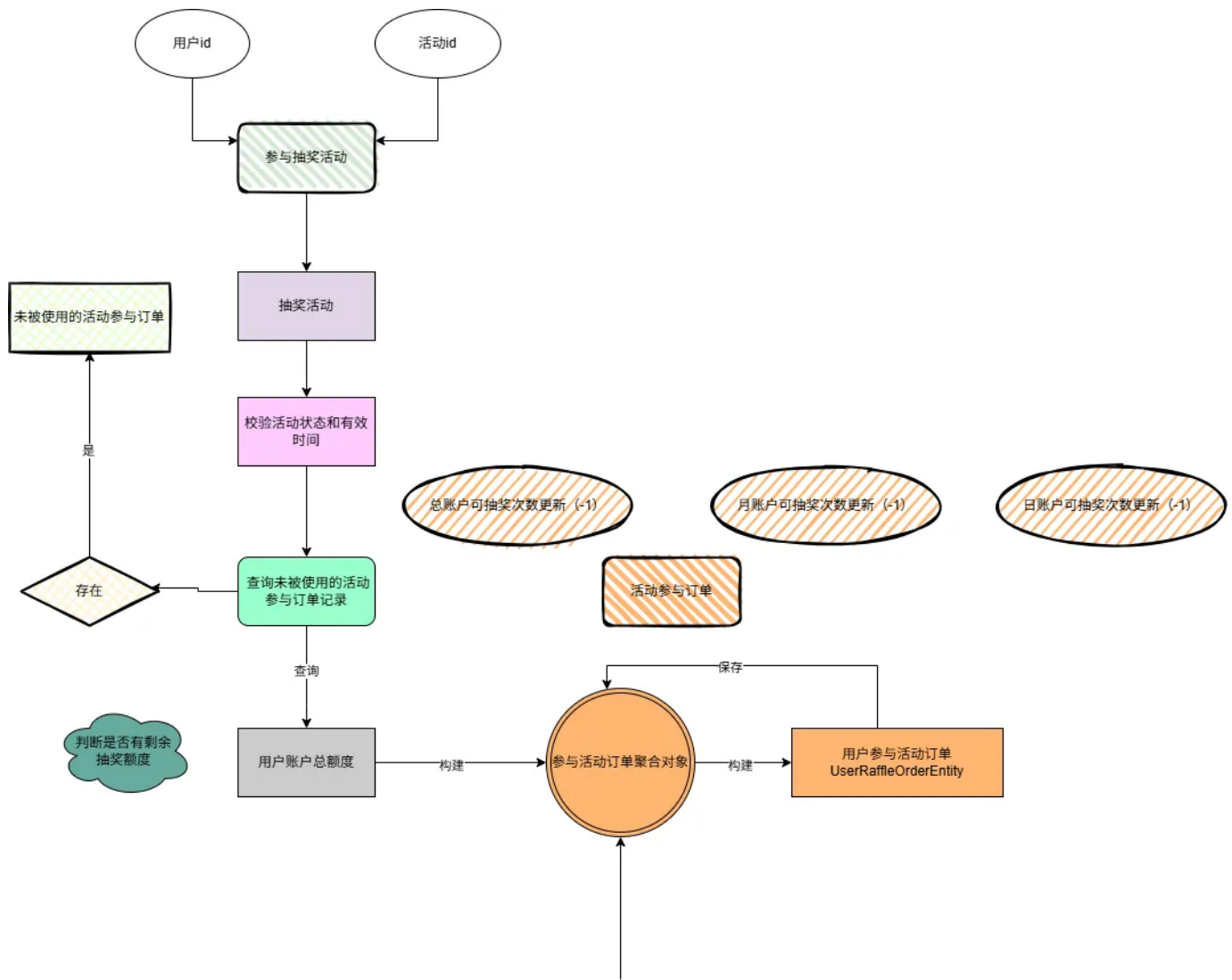


```
1  log.info("规则过滤-库存扣减 userId:{} strategyId:{} awardId:{})", userId, strategyId, awardId);
2  // 扣减库存
3  Boolean status = strategyDispatch.subtractionAwardStock(strategyId, awardId, endDateTime);
4  // true; 库存扣减成功, TAKE_OVER 规则节点接管, 返回奖品ID, 奖品规则配置
5  if (status) {
6      log.info("规则过滤-库存扣减-成功 userId:{} strategyId:{} awardId:{})", userId, strategyId, awardId);
7
8      // 写入延迟队列, 延迟消费更新数据库记录。【在trigger的job; UpdateAwardStockJob 下消费队列, 更新数据库记录】
9      strategyRepository.awardStockConsumeSendQueue(StrategyAwardStockKeyV0.builder()
10                                                     .strategyId(strategyId)
11                                                     .awardId(awardId)
12                                                     .build());
13
14      // 注意; 根据数据库表中配置走不同的节点。目前数据库配置 ALLOW 是走到下一个节点。
15      return DefaultTreeFactory.TreeActionEntity.builder()
16             .ruleLogicCheckType(RuleLogicCheckTypeV0.TAKE_OVER)
17             .strategyAwardV0(DefaultTreeFactory.StrategyAwardV0.builder()
18                             .awardId(awardId)
19                             .awardRuleValue(ruleValue)
20                             .build())
21             .build();
22  }
23
24  // 如果库存不足, 则直接返回放行
25  log.warn("规则过滤-库存扣减-告警, 库存不足.userId:{} strategyId:{} awardId:{})", userId, strategyId, awardId);
26  return DefaultTreeFactory.TreeActionEntity.builder()
27         .ruleLogicCheckType(RuleLogicCheckTypeV0.ALLOW)
28         .build();
```

```
1 public Boolean subtractionAwardStock(String cacheKey, Date endDateTime) {
2     long surplus = redisService.decr(cacheKey);
3     if (surplus < 0) {
4         // 库存小于0, 恢复为0个
5         redisService.setAtomicLong(cacheKey, 0);
6         return false;
7     }
8     // 1. 按照cacheKey decr 后的值, 如 99、98、97 和 key 组成为库存锁的key进行使用。
9     // 2. 加锁为了兜底, 如果后续有恢复库存, 手动处理等, 也不会超卖。因为所有的可用库存key, 都被加锁了。
10    String lockKey = cacheKey + Constants.UNDERLINE + surplus;
11    Boolean lock = false;
12    if (null != endDateTime) {
13        long expireMillis = endDateTime.getTime() - System.currentTimeMillis() + TimeUnit.DAYS.toMillis(1);
14        lock = redisService.setNx(lockKey, expireMillis, TimeUnit.MILLISECONDS);
15    } else {
16        lock = redisService.setNx(lockKey);
17    }
18    if (!lock) {
19        log.info("策略奖品库存加锁失败 {}", lockKey);
20    }
21    return lock;
22 }
```

抽奖活动领域

用户参与抽奖活动（参与抽奖子域）

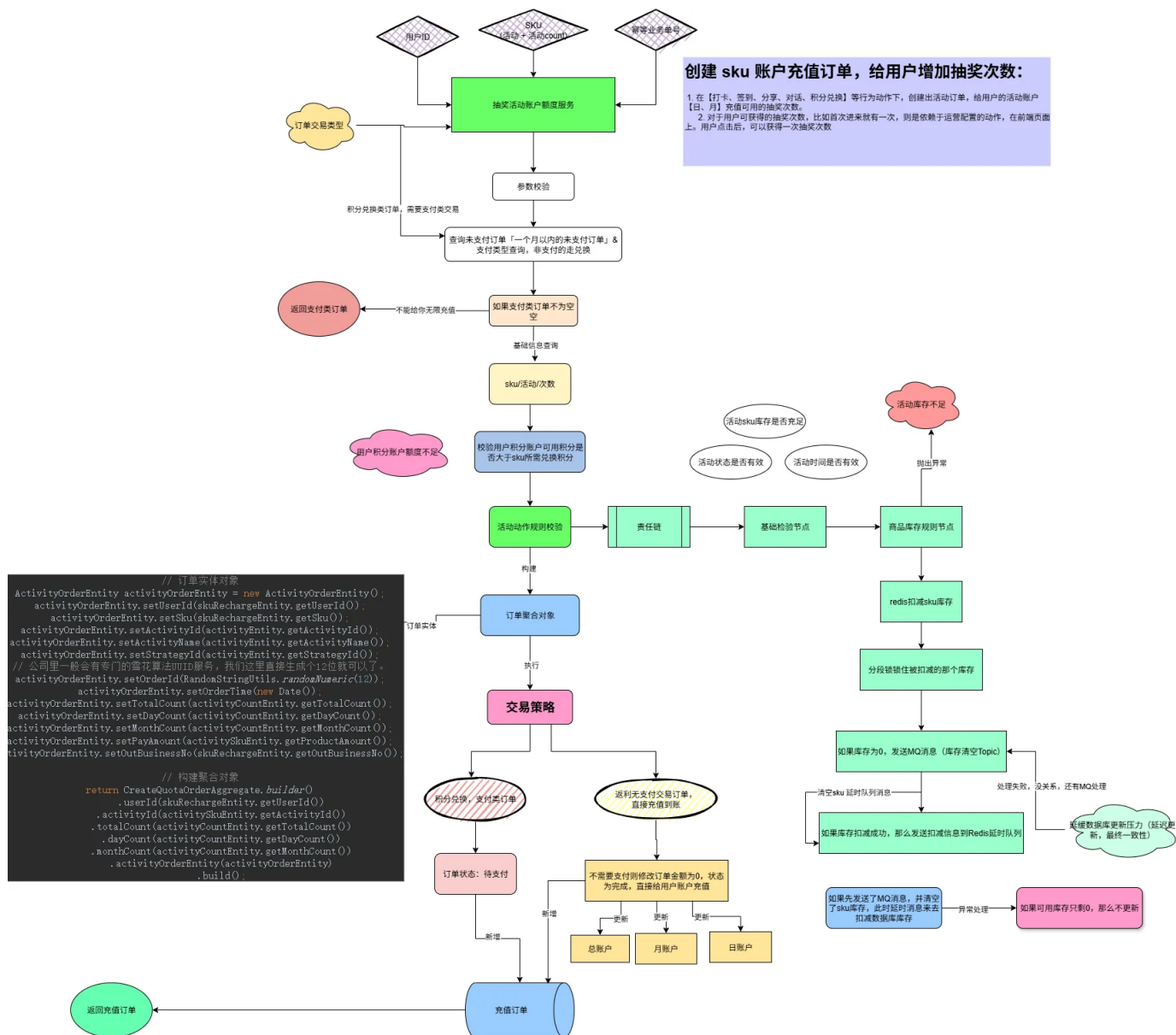


```

CreatePartakeOrderAggregate createPartakeOrderAggregate = new CreatePartakeOrderAggregate();
createPartakeOrderAggregate.setUserId(userId);
createPartakeOrderAggregate.setActivityId(activityId);
createPartakeOrderAggregate.setActivityAccountEntity(activityAccountEntity);
createPartakeOrderAggregate.setExistAccountMonth(isExistAccountMonth);
createPartakeOrderAggregate.setActivityAccountMonthEntity(activityAccountMonthEntity);
createPartakeOrderAggregate.setExistAccountDay(isExistAccountDay);
createPartakeOrderAggregate.setActivityAccountDayEntity(activityAccountDayEntity);
  
```

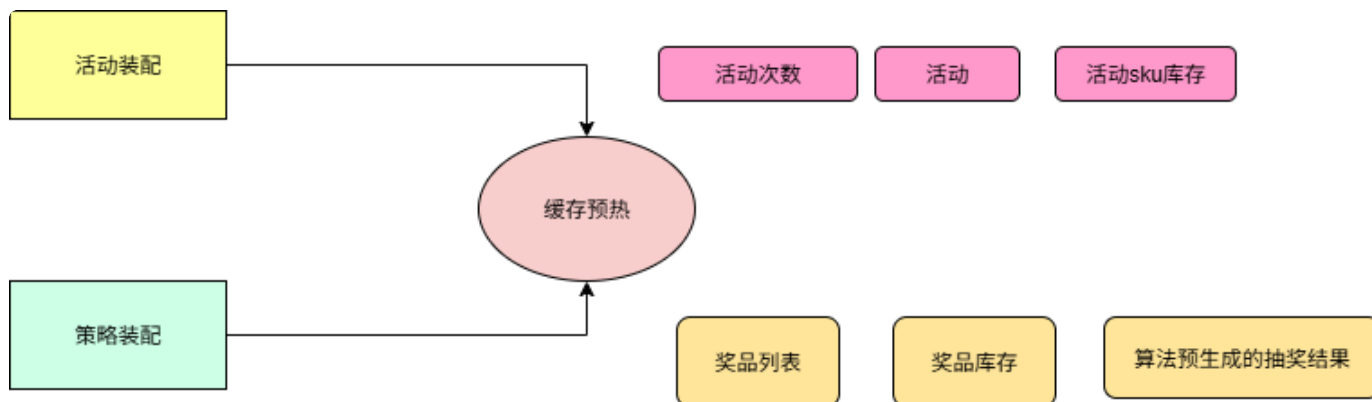
1. 如果存在活动参与订单，直接返回，不重复创建
2. 更新三个额度账户（月、日、总）的可参与抽奖次数（-1），同时新增活动参与订单

抽奖活动充值（额度子域）



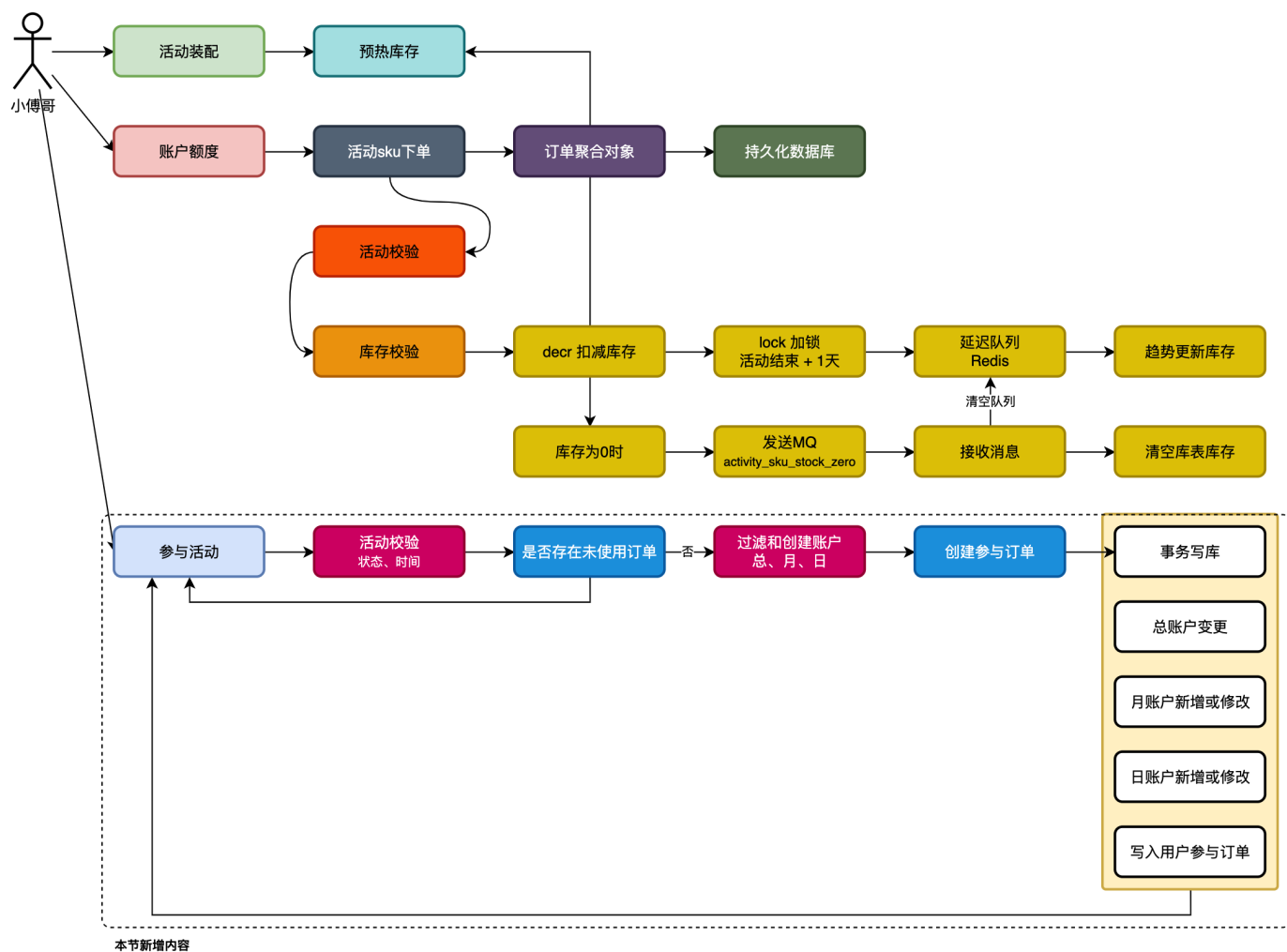
- 扣减活动sku库存（Redis扣减 + 分段锁 + 延时消息 + RocketMQ消息）
- 支付交易订单（如果存在，不会重复创建）+ 总账户 + 月账户 + 日账户、

活动装配子域（预热活动）



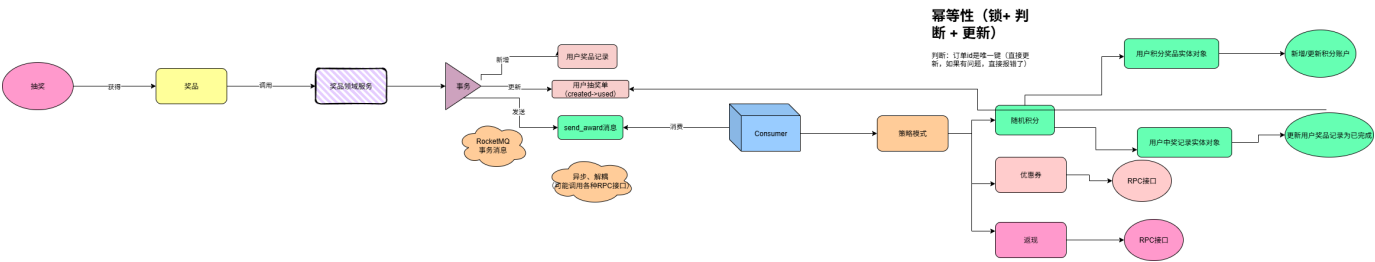
活动领域下各子域分析

作者画的很好了，就不重复画了



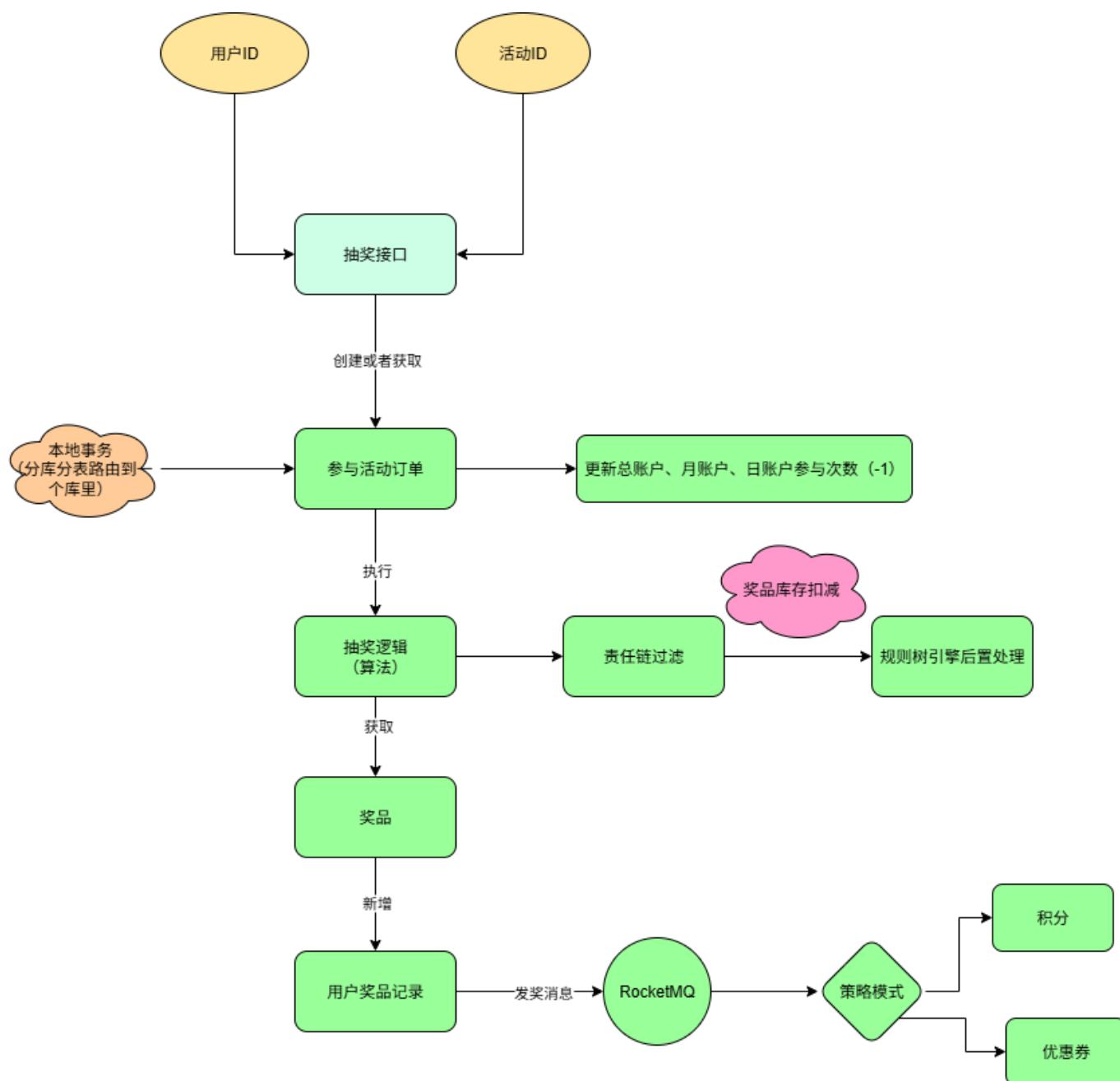
发奖领域

- 1. 策略模式
- 2. MQ事务消息（解耦、提升性能）



跨领域核心逻辑

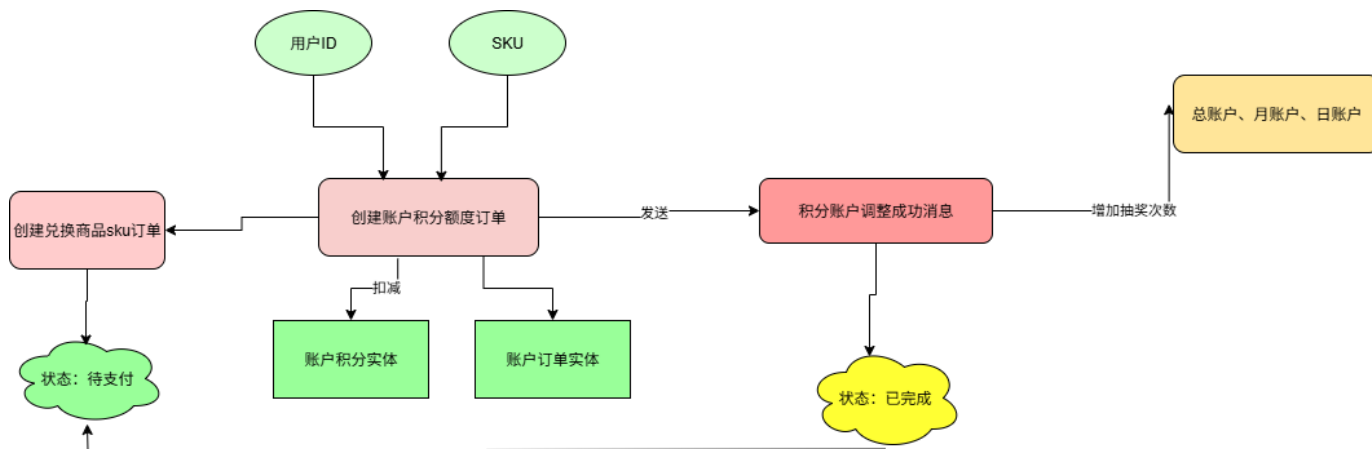
抽奖



返利领域

日常的返利会根据用户所完成的行为动作来触达，这包括；打卡、签到、连签、支付、开户、交易、信贷、拉新等类的动作。

- 打卡、签到、连签就是直接调用我们的RPC接口（同步）
- 支付、开户、交易、信贷 等需要解耦的业务，通常可以通过监听MQ消息来实现（异步）



1. 账户积分更新
2. 账户积分流水订单生成

